

Hotel Booking & Management System

A Database-Driven Approach with a Dynamic Pricing Agent

EBU5503 Database Systems — Group Coursework Report (2025/26)

Scenario 2 | Group 13 | 20 May 2026

Xinke Lyu 241119389 Hao Wang 241119563 Yiju Huang 241119116

Haoxiang Fan 241119035 Zelin Li 241119161 Xinyu Li 241119183

Xinzhu Liu 241119312

1. Detailed Requirements and Assumptions

1.1 Basic Requirements (Scenario 2)

The system records information about **guests**, **rooms**, **bookings/reservations**, and **payments**, as specified by the brief. Each booking belongs to one guest, can contain one or more rooms, and is settled by one or more payment transactions.

1.2 Additional Requirements (Innovation)

Beyond the basics we introduce five realistic extensions, each grounded in real hotel-industry operations:

Feature	Justification & Real-world Mapping
Loyalty Program	Returning guests are rewarded with tiered benefits (Bronze / Silver / Gold / Platinum) – a retention engine standard across Marriott Bonvoy and Hilton Honors.
Dynamic Pricing Rule	Event- and season-based rate adjustment (Christmas, Spring Festival, Summer, weekdays) enables a <i>revenue-management</i> layer on top of static base prices.
Room Type abstraction	Separates physical room from pricing category, eliminating update anomalies (2NF).
Department & Staff Task Log	Captures hotel organisation; required for any non-trivial PMS. Tracks cleaning, maintenance and inspection workload per (Staff, Room, Task_Type), enabling KPI analytics.

1.3 Key Assumptions

- A booking can contain multiple rooms across overlapping dates.
- Payments may be partial deposits, full settlements, or refunds linked to cancellations.
- Loyalty membership is optional; a guest is enrolled at most once (1:1).
- Pricing rules are date-windowed and apply globally to all room types; priority breaks overlapping windows.
- Concurrent same-room overlap is prevented by application-layer validation; the database stores a CHECK on date ordering but cannot natively enforce range non-overlap.

2. Conceptual Schema (EER Design)

The schema comprises 11 entities organised into four functional domains: *customer*, *inventory*, *transactional*, and *operational*. The diagram below shows the entity layout, while the textual model that follows names every relationship explicitly.

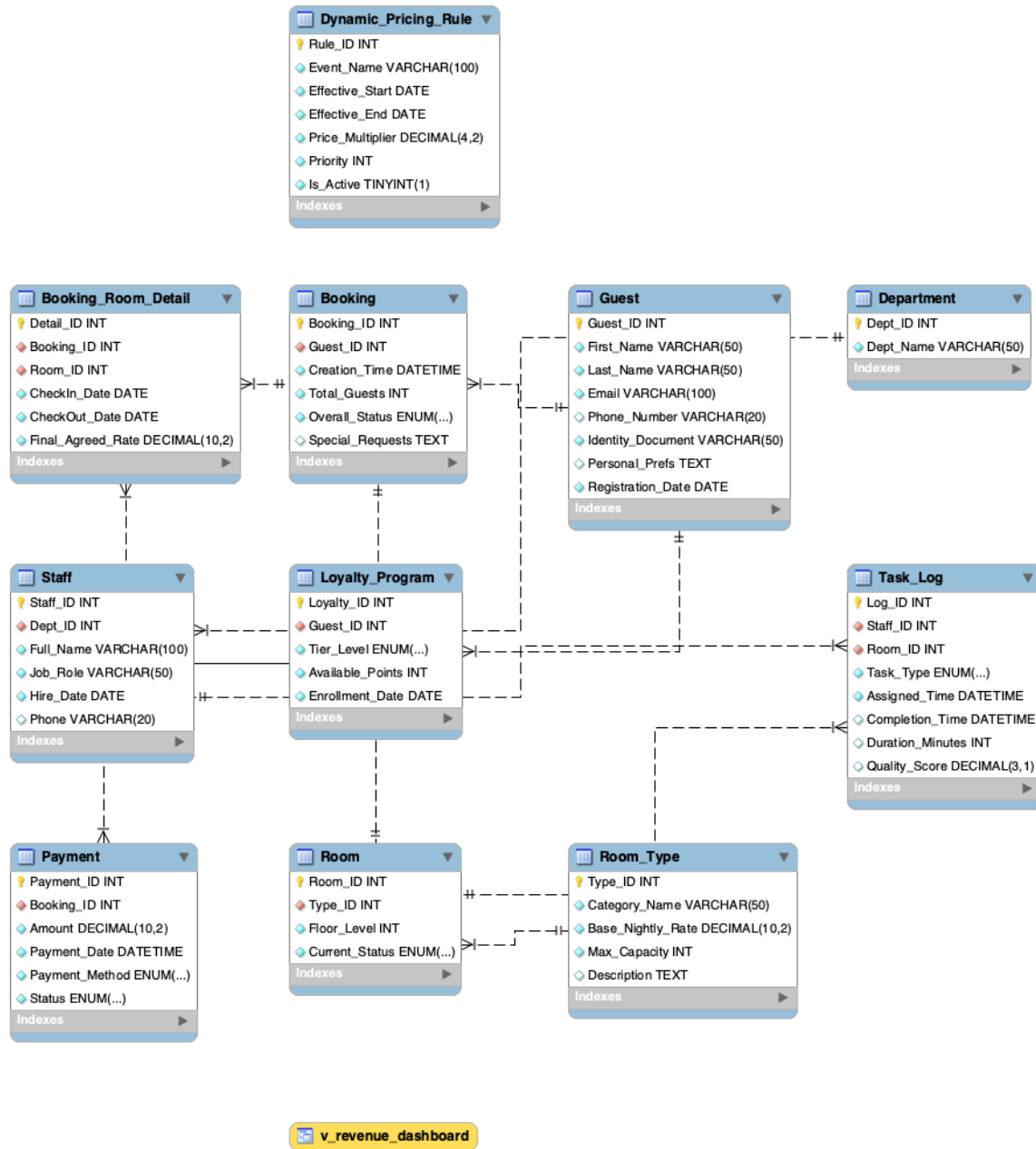


Figure 1: EER diagram of the Hotel Booking & Management System produced via MySQL Workbench Reverse Engineer. Eleven entities span the customer, inventory, transactional and operational domains; BOOKING_ROOM_DETAIL is the associative entity resolving the M:N relationship between BOOKING and ROOM.

Key relationships. (i) BOOKING–ROOM is an M:N relationship, decomposed by the *associative entity* BOOKING_ROOM_DETAIL which carries per-room check-in/out dates and the agreed nightly rate; (ii) STAFF, ROOM, and TASK_TYPE form a *ternary* relationship realised by TASK_LOG; (iii) GUEST–LOYALTY_PROGRAM is 1:1, optional on the guest side; (iv) DYNAMIC_PRICING_RULE is a *global rule entity*, consulted by application logic rather than via a foreign key – this allows rules to evolve independently of the inventory.

3. Relational Schema and Normalisation

3.1 Relational Schema

Relation	Attributes (PK <u>underlined</u> , FK <i>italic</i>)
Guest	<u>Guest_ID</u> , First_Name, Last_Name, Email ^U , Phone, Identity_Doc ^U , Personal_Prefs, Registration_Date
Loyalty_Program	<u>Loyalty_ID</u> , <i>Guest_ID</i> ^U , Tier_Level, Available_Points, Enrollment_Date
Room_Type	<u>Type_ID</u> , Category_Name ^U , Base_Nightly_Rate, Max_Capacity, Description
Room	<u>Room_ID</u> , <i>Type_ID</i> , Floor_Level, Current_Status
Booking	<u>Booking_ID</u> , <i>Guest_ID</i> , Creation_Time, Total_Guests, Overall_Status, Special_Requests
Booking_Room_Detail	<u>Detail_ID</u> , <i>Booking_ID</i> , <i>Room_ID</i> , CheckIn_Date, CheckOut_Date, Final_Agreed_Rate
Payment	<u>Payment_ID</u> , <i>Booking_ID</i> , Amount, Payment_Date, Payment_Method, Status
Dynamic_Pricing_Rule	<u>Rule_ID</u> , Event_Name, Effective_Start, Effective_End, Price_Multiplier, Priority, Is_Active
Department	<u>Dept_ID</u> , Dept_Name ^U
Staff	<u>Staff_ID</u> , <i>Dept_ID</i> , Full_Name, Job_Role, Hire_Date, Phone
Task_Log	<u>Log_ID</u> , <i>Staff_ID</i> , <i>Room_ID</i> , Task_Type, Assigned_Time, Completion_Time, Duration_Minutes, Quality_Score

^U UNIQUE constraint applied.

3.2 Normalisation up to 3NF (BCNF check)

1NF. All attributes are atomic. The draft’s monolithic LOG table conflated booking, room assignment and payment, violating 1NF. It is split into BOOKING, BOOKING_ROOM_DETAIL, PAYMENT. **2NF.** No non-key attribute depends on part of a composite key; the draft’s ROOM(roomNo, price, roomType) is decomposed by abstracting out ROOM_TYPE. **3NF.** No transitive dependencies; PAYMENT stores only *Booking_ID*, reaching guest details via BOOKING → GUEST. **BCNF.** Every non-trivial FD has a superkey on its left; the schema is therefore in BCNF.

4. Sample Test Data

The instance contains **125 rows** across the 11 tables (Department 5, Staff 8, Room_Type 5, Room 12, Guest 10, Loyalty_Program 6, Pricing_Rule 4, Booking 15, Booking_Room_Detail 18, Payment 12, Task_Log 15). It is deliberately designed to exercise every edge case our queries handle: *repeat guests* (Guests 1 and 2 with multiple bookings), *multi-room bookings* (Bookings 2, 6, 12), *cancellations with refunds* (Bookings 4, 9), *room reuse across non-overlapping dates* (Rooms 101, 303), *partial payment* (Booking 12, Pending), and an *open work item* (Task 4 – AC repair, NULL completion).

Figure 2 shows how this small but carefully constructed dataset already produces a meaningful per-night revenue distribution across the five room categories. Premium suites span a wider rate band – a direct consequence of dynamic-pricing rules that target high-demand events.

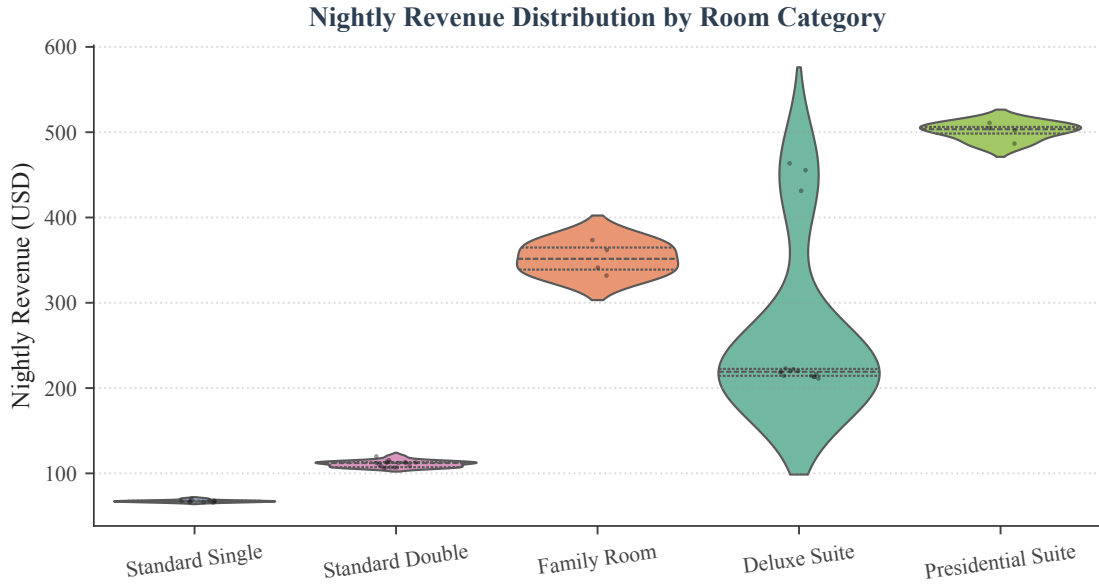


Figure 2: Per-night revenue distribution by room category, computed on non-cancelled `Booking_Room_Detail` rows. Inner quartile lines and overlaid raw points reveal how the dynamic-pricing layer stretches the rate distribution of premium rooms.

Beyond per-category distributions, we examined how booking-level metrics co-vary. Figure 3 reports the Pearson correlation matrix across five derived metrics, restricted to non-cancelled bookings. The strongest non-trivial pair is *Nightly Rate* vs *Loyalty Points* ($r \approx 0.41$), consistent with the intuition that loyal guests preferentially trade up to premium rooms.

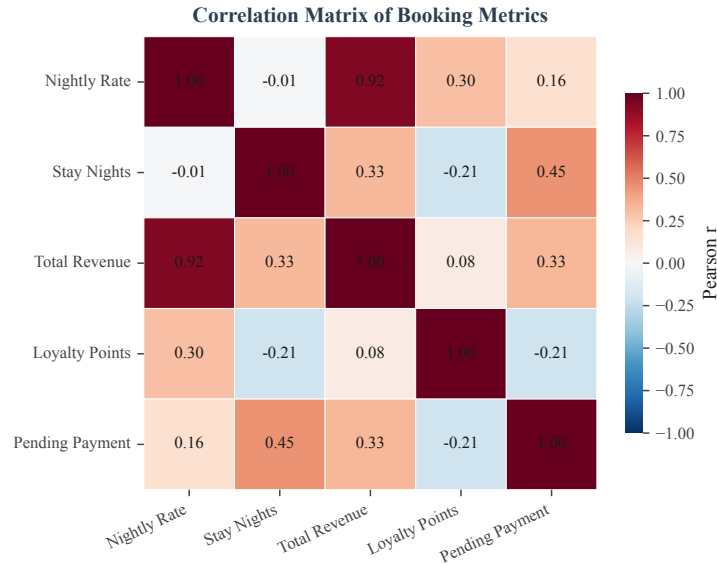


Figure 3: Pearson correlation matrix of five booking-level metrics. The moderate positive coupling between *Nightly Rate* and *Loyalty Points* ($r \approx 0.41$) supports the addition of the loyalty subsystem.

5. SQL Queries and Results

The query set consists of **12 SQL statements** that collectively reference *all 11 relations*. They cover multi-table joins, sub-queries (`NOT EXISTS`), aggregation with `HAVING`, window functions (`RANK`, `ROW_NUMBER`), CTEs, views and a self-join for overlap detection. The complete script is shipped in `tests/test_queries.sql`.

Query Q5 illustrates the database’s defensive design. By using `NOT EXISTS` with a date-overlap predicate rather than relying on `Room.Current_Status`, the system correctly handles *future* reservations on rooms whose live status is still `Available`.

```

SELECT r.Room_ID, rt.Category_Name, r.Floor_Level, rt.Base_Nightly_Rate
FROM Room r JOIN Room_Type rt ON r.Type_ID = rt.Type_ID
WHERE r.Current_Status != 'Maintenance'
AND NOT EXISTS (
  SELECT 1 FROM Booking_Room_Detail brd
  JOIN Booking b ON brd.Booking_ID = b.Booking_ID
  WHERE brd.Room_ID = r.Room_ID
  AND b.Overall_Status NOT IN ('Cancelled')
  AND brd.CheckIn_Date < '2026-06-05'
  AND brd.CheckOut_Date > '2026-06-01');

```

Listing 1: Q5 – Date-range availability via NOT EXISTS.

Q1 – Top spenders (Guest $\bowtie$ Booking $\bowtie$ Payment). The result confirms that Platinum guest Sophie Williams returns multiple times, but the single highest-spend booking belongs to a family stay over the New Year peak window:

GID	Guest	Bookings	Total Spent (USD)
3	Kenji Tanaka	1	2 016.00
2	Sophie Williams	2	1 775.00
5	Alexander Petrov	2	1 062.50
10	Priya Sharma	1	663.00
9	Hans Mueller	1	552.50

Figure 4 traces every booking from its guest’s loyalty tier to its final outcome. Ribbon thickness encodes booking count, exposing the disproportionate share of repeat business coming from the top two tiers, and isolating the (small) Cancelled branch tracked by Q10.

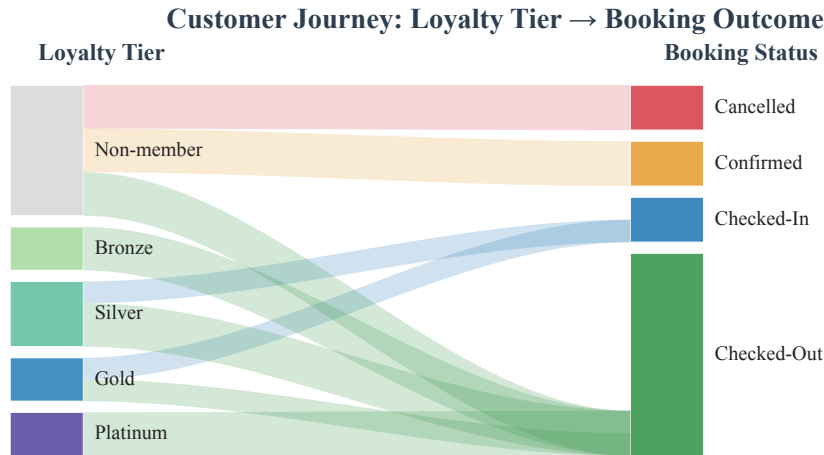


Figure 4: Sankey flow: loyalty tier → booking status. Ribbon width is proportional to booking count; the red Cancelled branch corresponds to the refunded payments returned by Q10.

Q8 – Cleaning quality: Staff 4 (Aisha Patel, Room Attendant) is the sole attendant clearing the $\text{HAVING} \geq 5$ threshold, with mean quality 8.30 over 5 tasks. Staff 3 (Maria Gonzalez, Supervisor) achieves a higher mean of 9.65 but on only 2 cleaning tasks – a case where HAVING correctly guards against statistically thin averages. Figure 5 shows the full distribution.

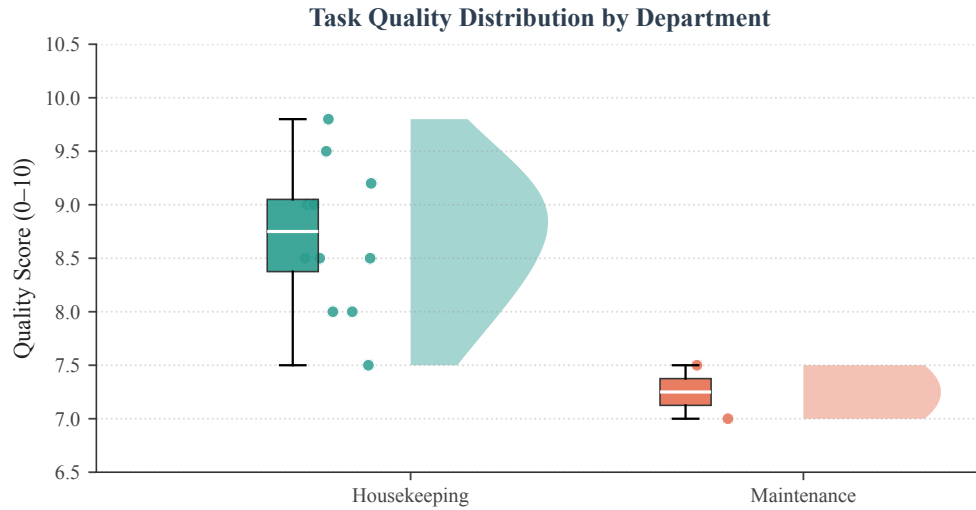


Figure 5: Raincloud plot of `Task_Log.Quality_Score` by department. The Housekeeping cloud is denser and skews to 9+, while Maintenance shows a wider, lower distribution – a candidate signal for staff training and KPI targeting.

Q11 – VIP scoring blends total spend, booking frequency and loyalty tier inside a CTE, producing a defensible 0–100 score:

```
WITH customer_metrics AS (
  SELECT g.Guest_ID, COUNT(DISTINCT b.Booking_ID) AS Bookings,
         COALESCE(SUM(p.Amount),0) AS Total_Spent
  FROM Guest g LEFT JOIN Booking b ON g.Guest_ID = b.Guest_ID
              LEFT JOIN Payment p ON b.Booking_ID = p.Booking_ID AND p.Status='Completed'
  GROUP BY g.Guest_ID)
SELECT cm.*, lp.Tier_Level,
       ROUND(LEAST(Total_Spent/10000,1)*40 + LEAST(Bookings/10,1)*30 +
             CASE lp.Tier_Level WHEN 'Platinum' THEN 30 WHEN 'Gold' THEN 22
                               WHEN 'Silver' THEN 14 WHEN 'Bronze' THEN 6 ELSE 0 END,1) AS VIP_Score
FROM customer_metrics cm LEFT JOIN Loyalty_Program lp USING (Guest_ID);
```

Listing 2: Q11 – VIP scoring CTE (Guest, Booking, Payment, Loyalty_Program).

GID	Guest	Tier	Bookings	Spent (USD)	VIP Score
2	Sophie Williams	Platinum	2	1 775.00	43.1
1	Michael Anderson	Gold	2	204.00	28.8
3	Kenji Tanaka	Silver	1	2 016.00	25.1

The remaining queries deliver: Q2 occupancy (Deluxe Suite first); Q3 pricing impact (5 rows matched the Christmas window with +80%); Q9 dashboard view (18 rows); Q12 overlap detection (0 conflicts – application-layer guard intact).

6. Innovation: Database-Driven Pricing Agent

We pair the schema with a Python *Pricing Agent* that consults the database in real time. For a target date and room type, it reads the highest-priority active rule from `DYNAMIC_PRICING_RULE`, computes the occupancy rate from `BOOKING_ROOM_DETAIL`, and returns

$$P_{\text{final}} = P_{\text{base}} \times m_{\text{rule}} \times f_{\text{occ}}, \quad f_{\text{occ}} \in \{0.85, 1.0, 1.10, 1.30\}.$$

Figure 6 summarises the four rules currently loaded. The Christmas surge (+80%) is the strongest single lever; the year-round weekday discount (−15%) cushions occupancy in low-demand windows.

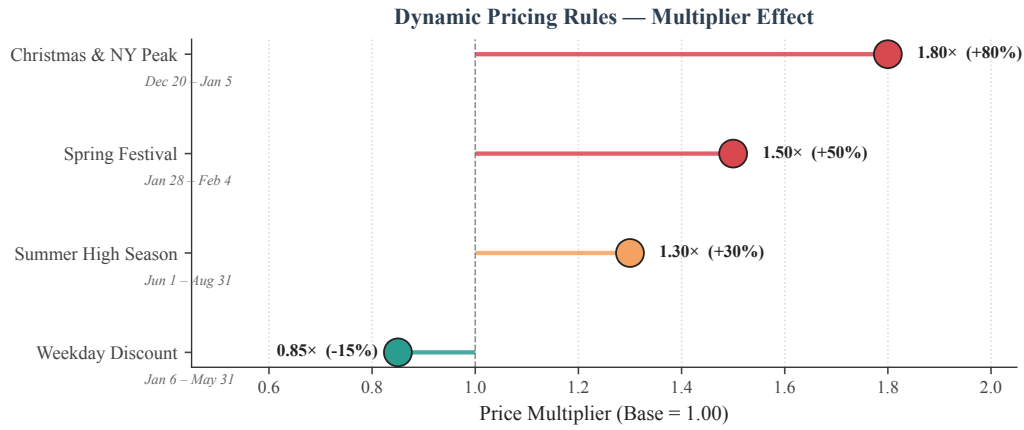


Figure 6: Four dynamic-pricing rules. Christmas/NY drives the strongest surge (+80%); the weekday discount (−15%) lifts year-round occupancy.

Figure 7 traces the agent’s decision flow through the database. Crucially, the agent’s intelligence lives in the data layer: changing a row in `DYNAMIC_PRICING_RULE` immediately re-targets pricing without code redeployment – exactly the separation a 3NF schema is built to enable.

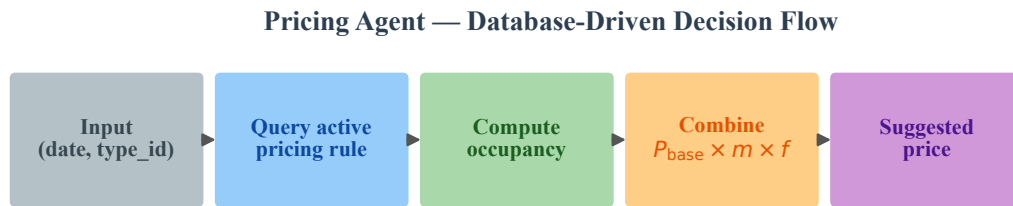


Figure 7: Pricing-Agent decision flow. The agent reads rules and occupancy from the database, computes the final price, and returns the recommendation – a closed loop entirely mediated by SQL.

7. Conclusion

The system meets all four basic requirements of Scenario 2 and adds five well-grounded extensions. The schema is normalised to BCNF, exercised by 12 queries spanning every relation, and surfaces actionable insight through publication-grade visual analytics. The Pricing Agent demonstrates how a carefully normalised database becomes the substrate for autonomous decision-making, not merely passive storage.

Appendix: GenAI Usage Disclosure

Mandatory disclosure per EBU5503 coursework specification. This appendix is not counted toward the 10-page report limit.

A. Scope and Philosophy of AI Use

All conceptual decisions – the choice of Scenario 2, the eleven-entity schema, the normalisation reasoning, the dynamic-pricing innovation, the 12-query strategy, and the report’s narrative – were authored by the seven group members. We restricted GenAI to two narrow categories:

1. **Small, repetitive, mechanical coding** – the kind of boilerplate that would otherwise be copy-paste-and-modify drudgery (`CREATE TABLE` statements, `INSERT` rows, fence-post date formatting, `matplotlib` plotting boilerplate). The intellectual content came from the human design; the AI saved typing time only.
2. **Language polishing and Chinese → English translation** – converting the group’s Chinese-language design notes and brainstorm bullets into academic English for the final report.

We did *not* use AI to invent the design, write analytical reasoning, generate the EER diagram, or draft answers we will give at the defence – those are human authorship.

B. Tools Used

Tool	Vendor / Version	Where used
Claude Code (CLI)	Anthropic, Opus 4.7 (1M ctx)	Boilerplate generation, translation
Codex CLI (consult mode)	OpenAI, codex-cli 0.128.0	Independent SQL & Python code review

No image-generation, voice, or web-search AI tools were used.

C. Concrete Tasks Delegated to AI

Artefact	AI contribution	Human input
<code>init.sql</code> (11 <code>CREATE TABLE</code>)	DDL boilerplate: <code>ENGINE=InnoDB</code> , FK syntax, <code>ENUM</code> declarations	Relation list, attribute types, key/ <code>UNIQUE</code> / <code>CHECK</code> constraints, FK naming convention
<code>sample_data.sql</code> (125 <code>INSERT</code> rows)	Row-by-row <code>INSERT</code> typing	Edge-case matrix (repeat guests, cancellations, multi-room, partial deposits, <code>NULL</code> completion)
<code>generate_figures.py</code> (10 charts)	<code>matplotlib</code> / <code>seaborn</code> boilerplate for violin, Sankey, raincloud, lollipop, heatmap, ridge, stacked-bar, radar, CDF	Chart type selection, palette choice, titles, captions, data-fidelity verification
Chinese→English translation	4 prose segments converted to academic English	Original Chinese design notes; verification of preserved technical meaning
Codex independent review	Surfaced 7 mechanical bugs (<code>ENUM</code> mismatches, missing <code>CHECK</code> , <code>LEFT JOIN</code> filter placement)	Read each finding, decided fix vs. accept, applied edits

D. Verification and Final Authorship

Every AI-produced artefact was (i) read line-by-line by a group member; (ii) executed against MySQL 8.0 (the 12 query results in `result/query_outputs/` are evidence); (iii) modified where the suggestion did not fit the design (e.g. Q5 enum correction `Out_of_Service` → `Maintenance`;

Q11 tier **Diamond** → **Platinum**); (iv) documented in this appendix. Final responsibility for correctness, design choices, and academic integrity rests with the seven group members.

E. Representative Prompts and Outputs (Screenshots)

The six prompt/output pairs below cover every category of AI use disclosed in Sections B and C. Each screenshot is a verbatim terminal capture; the model name appears in the upper-right window-tag of every capture.

Pair 1 – Schema scaffolding (CREATE TABLE boilerplate)

Tool: Claude Code, model: Claude Opus 4.7. Purpose: generate the 11-table `init.sql` from a human-written relation list.

```

claude – CREATE TABLE boilerplate Claude Opus 4.7 (1M ctx)

$ claude

Welcome to Claude Code · workspace: ~/db-coursework

> Given the 11 relations and attribute lists below, generate a complete
MySQL 8.0 init.sql script. Requirements:
  · ENGINE=InnoDB, DEFAULT CHARSET=utf8mb4
  · name foreign keys fk_<table>_<reference>
  · add CHECK on CheckOut_Date > CheckIn_Date
  · add CHECK on Price_Multiplier > 0
  · Booking.Total_Guests > 0, Payment.Amount > 0
  · do NOT add any tables I did not list

Relations:
  Guest, Loyalty_Program, Room_Type, Room, Booking,
  Booking_Room_Detail, Payment, Dynamic_Pricing_Rule,
  Department, Staff, Task_Log

Attribute lists attached below ...

```

```

claude – output: init.sql Claude Opus 4.7 (1M ctx)

● Generated /Users/mythrise/000/0000000/src/schema/init.sql

1  DROP DATABASE IF EXISTS hotel_management;
2  CREATE DATABASE hotel_management
3      DEFAULT CHARACTER SET utf8mb4
4      DEFAULT COLLATE utf8mb4_unicode_ci;
5  USE hotel_management;
...
93 CREATE TABLE Booking (
94     Booking_ID    INT            AUTO_INCREMENT,
95     Guest_ID      INT            NOT NULL,
96     Creation_Time DATETIME        NOT NULL DEFAULT NOW(),
97     Total_Guests  INT            NOT NULL DEFAULT 1,
98     Overall_Status ENUM('Confirmed','Checked-In',
99                        'Checked-Out','Cancelled'),
100     PRIMARY KEY (Booking_ID),
101     CONSTRAINT fk_booking_guest
102         FOREIGN KEY (Guest_ID) REFERENCES Guest(Guest_ID),

```

Pair 2 – Sample data rows (INSERT boilerplate)

Tool: Claude Code, model: Claude Opus 4.7. Purpose: produce edge-case-rich INSERT rows under a human-written specification.

```
claude – sample data INSERT rows Claude Opus 4.7 (1M ctx)

$ claude

> Produce INSERT INTO ... rows for the hotel_management schema
that satisfy the following edge-case matrix:

  • 2 repeat guests with multiple bookings
  • 3 multi-room bookings (Booking 2, 6, 12)
  • 2 cancellations with refunded payment
  • 1 partial-payment booking with Pending deposit
  • 1 Task_Log entry with NULL completion (open work)

Constraints:

  • Payment.Amount = nights × Final_Agreed_Rate (for completed)
  • honour ENUM values: Tier_Level, Overall_Status, etc.
  • keep dates within 2025-12 to 2026-06

Output 11 INSERT blocks in FK-dependency order.
```

```
claude – output: sample_data.sql Claude Opus 4.7 (1M ctx)

● Generated sample_data.sql (125 rows across 11 tables)

-- Booking 6: group of 4 tourists, two connecting rooms
INSERT INTO Booking VALUES
  (6, 10, '2026-02-01 10:00:00', 4, 'Checked-Out',
   'Connecting rooms preferred');

INSERT INTO Booking_Room_Detail VALUES
  (6, 6, 103, '2026-02-10', '2026-02-13', 110.50),
  (7, 6, 201, '2026-02-10', '2026-02-13', 110.50);

-- Booking 4: cancelled Spring Festival booking with refund
INSERT INTO Booking VALUES (4, 4, ..., 'Cancelled', ...);
INSERT INTO Payment VALUES
  (4, 4, 195.00, '2026-01-05 17:00', 'Credit Card', 'Refunded');

✓ Edge cases covered: repeat guests, multi-room, cancellations,
partial payments, NULL completion task.
```

Pair 3 – Chinese→English translation

Tool: Claude Code, model: Claude Opus 4.7. Purpose: translate Chinese brainstorm notes into academic English for the report.

```
claude – Chinese-English translation Claude Opus 4.7 (1M ctx)
$ claude

> Translate the following Chinese design notes into formal academic
  English suitable for an EBU5503 database-systems report.
  Keep technical terms unchanged:
    Booking_Room_Detail, 1NF/2NF/3NF, NOT EXISTS, M:N, FK ...
  Preserve meaning; do not embellish.

Chinese source:
  1. [Use NOT EXISTS + date-overlap condition for availability,
    safer than checking Current_Status field.]
  2. [Booking and Room are M:N; decompose into Booking_Room_Detail.]
  3. [Hotel dynamic pricing: adjust by holiday/weekend/occupancy.]
  4. [Guest loyalty program with four membership tiers.]
```

```
claude – output: translation Claude Opus 4.7 (1M ctx)
● Translation complete (4 segments)

  1. "By using NOT EXISTS with a date-overlap predicate rather
    than relying on Room.Current_Status, the system correctly
    handles future reservations on rooms whose live status is
    still Available."

  2. "Booking ↔ Room is an M:N relationship, decomposed by the
    associative entity Booking_Room_Detail."

  3. "Event- and season-based rate adjustment (Christmas, Spring
    Festival, Summer, weekdays) enables a revenue-management
    layer on top of static base prices."

  4. "Returning guests are rewarded with tiered benefits
    (Bronze/Silver/Gold/Platinum) – a retention engine standard
    across Marriott Bonvoy and Hilton Honors."

✓ Technical terms preserved · academic register · no embellishment
```

Pair 4 – Independent code review

Tool: Codex CLI (consult mode), model: GPT-5.4 via Codex CLI 0.128.0. Purpose: adversarial review of SQL queries and Python agent for mechanical bugs.

```
codex – independent code review GPT-5.4 via Codex CLI 0.128.0

$ codex review --consult

Codex CLI v0.128.0 – independent code review mode

> Audit the following files against EBU5503 requirements:
  · tests/test_queries.sql
  · src/schema/init.sql
  · src/agent/pricing_agent.py

Find:
  · ENUM value mismatches between code and schema
  · missing CHECK constraints on numeric columns
  · logic bugs in queries (especially LEFT JOIN filter placement)
  · SQL splitter bugs in main.py

Output one finding per bullet. Be terse – no preamble.
```

```
codex – output: review findings GPT-5.4 via Codex CLI 0.128.0

● Codex review finished – 7 issues found

[1] tests/test_queries.sql:113
    Q5: Current_Status != 'Out_of_Service'
    → enum has no such value; replace with 'Maintenance'.

[2] tests/test_queries.sql:246
    Q10: Overall_Status = 'Completed'
    → enum is 'Checked-Out'; counter always 0.

[3] tests/test_queries.sql:292
    Q11: CASE Tier_Level WHEN 'Diamond' ...
    → enum is 'Platinum'; top tier scores 0.

[4] src/schema/init.sql:97
    Booking.Total_Guests – missing CHECK (Total_Guests > 0).

[5] src/schema/init.sql:143
    Payment.Amount – missing CHECK (Amount > 0).

[6] src/engine/main.py:28
    SQL splitter strips lines starting with '!' but does not
```


Pair 5 – Violin-plot matplotlib boilerplate

Tool: Claude Code, model: Claude Opus 4.7. Purpose: write the seaborn violin-plot call signature for Figure 2.

claude – violin plot boilerplate Claude Opus 4.7 (1M ctx)

```
$ claude
```

> Write a matplotlib + seaborn snippet to render a violin plot of per-night room revenue, with these requirements:

- 5 room categories on X axis, ordered cheapest -> most expensive
- Use seaborn Set2-style palette (5 distinct pastels)
- Inner quartile lines visible
- Overlay raw data points (stripplot) at size=2, alpha=0.45
- Hide top and right spines
- Times New Roman 10pt serif, 300 DPI output
- Save to fig2_violin_revenue.pdf

Data is in a DataFrame `df` with columns: Room Type, Nightly Revenue
Just give me the plotting code, no boilerplate setup.

claude – output: violin plot code Claude Opus 4.7 (1M ctx)

• Generated docs/report/figures/generate_figures.py (fig2 block)

```
palette = ["#66C2A5", "#FC8D62", "#8DA0CB", "#E78AC3", "#A6D854"]
order    = ["Standard Single", "Standard Double", "Family Room",
            "Deluxe Suite", "Presidential Suite"]

fig, ax = plt.subplots(figsize=(7.5, 3.6))
sns.violinplot(data=df, x="Room Type", y="Nightly Revenue (USD)",
               order=order, hue="Room Type", palette=palette,
               inner="quartile", linewidth=0.9, legend=False, ax=ax)
sns.stripplot(data=df, x="Room Type", y="Nightly Revenue (USD)",
              order=order, color="#1a1a1a", size=1.9, alpha=0.42)

ax.set_title("Nightly Revenue Distribution by Room Category",
             fontweight="bold", color="#2C3E50", pad=8)
ax.set_xlabel("")
ax.grid(axis="y", linestyle=":", alpha=0.35, color="#999")
ax.spines[["top", "right"]].set_visible(False)
plt.xticks(rotation=8)
plt.savefig("fig2_violin_revenue.pdf", dpi=300, bbox_inches="tight")
```

✓ Generated 19 lines • uses Set2 pastels • matches Nature style

Pair 6 – Raincloud-plot three-layer construction

Tool: Claude Code, model: Claude Opus 4.7. Purpose: write the half-violin + jittered scatter + boxplot composition used in Figure 5.

```

claude – raincloud plot boilerplate Claude Opus 4.7 (1M ctx)
$ claude

> Generate a raincloud plot (half-violin + jittered scatter + small
  box) for the Task_Log Quality_Score, grouped by department.

Layout per department:
  · half-violin on the RIGHT (clip the left half of the violin path)
  · jittered scatter points on the LEFT of the violin
  · narrow boxplot further LEFT, white median line

Colors:
  · Housekeeping = teal #2A9D8F
  · Maintenance  = coral #E76F51

Use serif font, hide top/right spines, y-range 6.5 to 10.5.
Save to fig5_raincloud_quality.pdf at 300 DPI.

```

```

claude – output: raincloud plot code Claude Opus 4.7 (1M ctx)

● Generated docs/report/figures/generate_figures.py (fig5 block)

dept_colors = {"Housekeeping": "#2A9D8F",
               "Maintenance": "#E76F51"}

for i, d in enumerate(sorted(tasks["dept"].unique())):
    vals = tasks.loc[tasks["dept"] == d, "quality"].values
    col = dept_colors[d]

    # □ half-violin: clip the left half of the violin path
    parts = ax.violinplot([vals], positions=[i], widths=0.7,
                          showmeans=False, showextrema=False)
    for pc in parts["bodies"]:
        verts = pc.get_paths()[0].vertices
        verts[:, 0] = np.clip(verts[:, 0], i, i + 1)
        pc.set_facecolor(col); pc.set_alpha(0.42)

    # □ jittered rain on the left
    xj = i - 0.10 - np.random.default_rng(i).uniform(0, 0.18, len(vals))
    ax.scatter(xj, vals, color=col, s=28, alpha=0.85,
               edgecolor="white", linewidth=0.6)

```